AI Solver for 2048 Game



Table of Contents

I. Introduction..... 3
II. Background..... 3
III. Methodology..... 3
    A. Game representation..... 3
    B. Expectimax..... 4
    C. Heuristics & Evaluation Function..... 4
    D. Computational Complexity..... 6
    E. Tools and Libraries..... 7
IV. Results..... 7
V. Discussion..... 7
VI. Conclusion..... 8

I. Introduction

2048 was built by Gabriele Cirulli, who set out to challenge himself with a simple coding project and ended up creating one of the most addictive puzzle games in the world. The goal and rules of the game are straightforward. 2048 is a single-player sliding tile puzzle game where the main objective is to combine matching tiles to create an even bigger number. The game consists of a 4x4 grid and two randomly placed tiles, with the numbers 2 or 4, that are able to be slid in four directions: up, down, left, or right. The tiles will move as far as possible in that direction, either until they merge with a tile of the same value or are stopped by another tile with a different value or the border. After the move completes a random tile, with the numbers 2 or 4, will appear on the grid. Once there are no empty tiles and no more valid moves available, the game ends. If a tile with the number 2048 is on the grid, then the player wins. If not, they lose. The scoreboard starts at 0 and increases when tiles are merged together. Our goal for this project was to build an AI that could play the game and get the highest score. 2048, due to its simple yet complex nature has allowed for strategies to form like picking a corner for higher values, keeping an active row filled with tiles, or merging the smaller tiles first. Just like these strategies evolved over time to help players beat the game, we wanted to explore which algorithms could be used to further exploit the game's mechanics and maximize the score.

II. Background

This project relies on both the mechanics of the 2048 game and the algorithms that can optimize decision making in such environments. Considering the possible states of the game, randomness of the number being displayed (either a 2 or 4), and the different placement of tiles, Expectimax seems like the ideal algorithm to use. Expectimax is a variation of the minimax algorithm that takes into account chance elements. Unlike minimax, which assumes the best or worst-case scenario for each move, Expectimax evaluates all the possible outcomes and adjusts the decision tree accordingly. It works by building decision trees representing nodes of a potential game state and each branch is a move or random tile placement. This allows it to make a more informed decision about which moves will lead to a higher score and prevent undesirable configurations. When taking into consideration the random tile placements, Expectimax can plan multiple steps ahead and minimize the risk of running losing.

III. Methodology

A. Game representation

The game board was represented as a 4x4 grid where each cell held a tile with a value of 2^i where $i \in \{1,2,\ldots,17\}$. The action space consisted of four possible moves—up, down, left, and right—that allowed tiles to slide, merge if they had the same value, and generate new random tiles (2 or 4). The transition function defined the game's mechanics, including tile movement, merging logic, and the placement of new tiles.

B. Expectimax

To handle the randomness nature of 2048, we implemented the Expectimax algorithm, an extension of Minimax that accounts for probabilistic events. Expectimax modeled the game as a two-player simulation: the AI agent (player) made decisions to maximize the score by moving the tiles, while the computer (opponent) represented the random tile placements of 2 or 4. AI decisions were represented as decision nodes, and random tile placements as chance nodes with probabilities 90% for 2, 10% for 4. Fig. 1 shows the game tree that the algorithm uses.

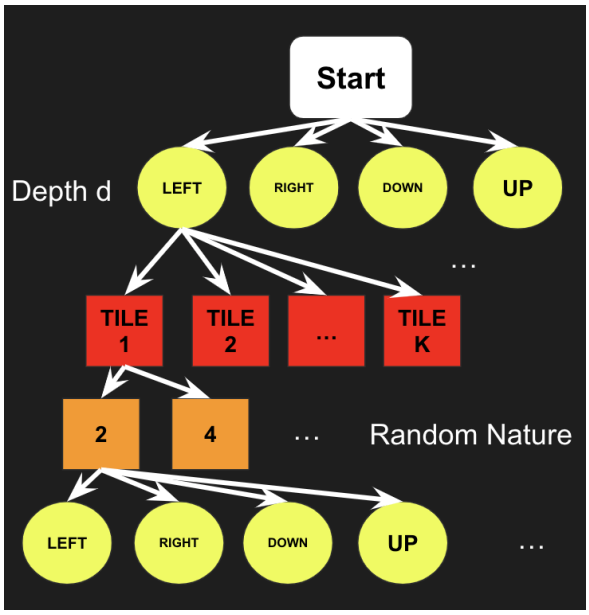


Figure 1: Game Tree

C. Heuristics & Evaluation Function

The evaluation function plays a crucial role in guiding the AI's decisions by assigning a numerical value to different board states based on the heuristics used. Four key heuristics were employed to evaluate the state of the board and inform the AI's actions: the number of empty tiles, the snake pattern, the adjacent tile heuristic, and the largest tile heuristic (which was initially considered but ultimately not used). The total evaluation score is calculated as:

$$Evaluation = (0.5 \times Empty\ Tiles) + (1.5 \times Snake\ Pattern) + (1 \times Adjacent\ Tile)$$

1. The Empty Tiles heuristic on maximizing the number of unoccupied tiles on the board is important because it provides flexibility for future moves and reduces the chance of a game-over by preventing a cluttered board. By evaluating potential moves based on the number of tiles freed, this heuristic prioritizes space optimization over multiple turns. It is integrated into the Expectimax algorithm to encourage sustainable gameplay by maintaining an adaptable board state. This approach helps balance tile merging with space utilization, allowing the AI to make more strategic decisions and avoid running out of valid moves.

2. The snake heuristic arranges tiles in a specific "snake" order, where higher-value tiles are placed towards one end of the board, and lower-value tiles are positioned in a descending order toward the other end. This ordering is achieved by increments of 4^i in a snake pattern, with the largest value pushed into a corner and the decreasing values following a structured order. The heuristic is composed of several components: the snake pattern score, an empty tile bonus, a mergeable tiles bonus, and a penalty for deviating from the snake pattern. The snake pattern score, which carries the highest weight, rewards the AI for positioning tiles according to a weighted matrix. The empty tile bonus incentivizes configurations with more open spaces, while the mergeable tiles bonus rewards tiles that are adjacent to each other and can potentially be merged. The penalty discourages the AI from creating disorganized or inefficient tile placements. Additionally, the heuristic calculates the sumproduct of the weight matrix and the game state to determine the overall quality of the board arrangement, ensuring that the AI prioritizes efficient tile placements that support future moves and merges.
3. The Adjacent Tile heuristic prioritizes positions near the corners of the grid, where tiles with similar values should be placed for optimal gameplay. It assigns higher weights to the corners, with values decreasing inward. The heuristic is computed as the sum of the products of the board state and the weight matrix, encouraging the placement of higher-value tiles in the corners to facilitate better merging and controlled gameplay.
4. We initially worked on integrating the largest tile heuristic, which evaluates the value of the highest tile on the board. The idea was that the larger the highest tile, the closer the AI would be to achieving its goal, as large tiles often indicate successful merges and a well-organized board. However, after further consideration and experimentation, we decided not to use this heuristic in our project as this heuristic wasn't considering the board on a broader level, thus impacting the performance of our evaluation function. We can also notice that in Fig. 2, our algorithm achieves 2048 more often using three heuristics compared to four heuristics, as shown in Fig. 3.

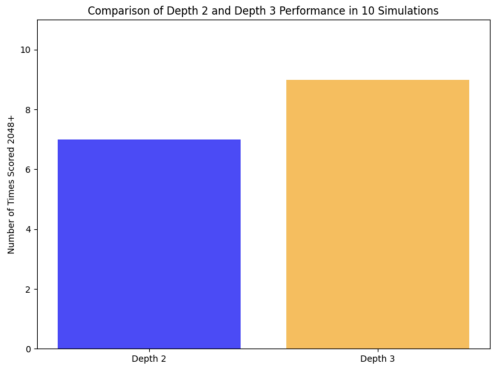


Figure 2: Using 3 Heuristics

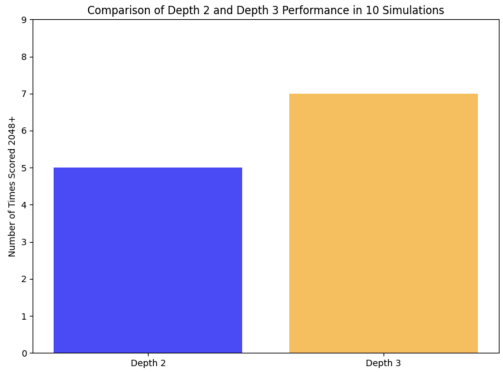


Figure 3: Using 4 Heuristics

D. Computational Complexity

The computational complexity of the AI is significantly influenced by the choice of depth in the Expectimax algorithm. Depth determines how far ahead the AI looks to evaluate potential moves, with deeper levels providing a more informed evaluation of future states but requiring significantly more computation. At each level of depth, the branching factor grows due to the four possible moves (up, down, left, right) and the subsequent probabilistic placement of tiles (90% chance for a 2 and 10% chance for a 4). This exponential growth makes depth selection a critical factor in balancing computational feasibility and decision quality. Additionally, we prioritized heuristics that could be computed quickly, ensuring that the evaluation function remained efficient even as the depth increased.

To determine the optimal depth, we experimented with different levels assessing both AI performance and computation time. Higher depths allowed the AI to predict better moves by evaluating long-term outcomes, but increased computation time significantly. Depth 2, while faster with an average computation time of 22 seconds, evaluated fewer outcomes, leading to missed opportunities, poor long-term planning, and rarely achieving scores of 4096 or higher. Depth 3 emerged as the optimal depth, with an average computation time of 212 seconds, striking a balance between performance and speed. It consistently produced better results, achieving scores of 4096 or higher more often while evaluating 64 states to make informed decisions. Depth 4, although capable of evaluating deeper outcomes, had an average computation time exceeding 45 minutes, making it impractical for efficient gameplay. Fig. 4 shows a comparison between depth and the average time of the algorithm, illustrating the trade-off between evaluation depth and computational efficiency.

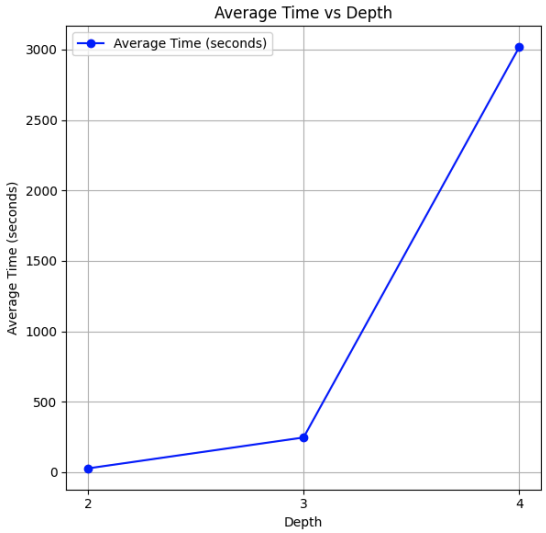


Figure 4: Comparison of Algorithm Performance and Computation Time at Different Depths

E. Tools and Libraries

The project was developed using Google Colab for experimentation and debugging due to its cloud-based computational capabilities. Git was employed for version control and collaboration, ensuring efficient tracking of code changes and team contributions. The implementation relied on Python libraries such as NumPy for matrix operations, random for simulating tile placements, and tabulate for visualizing board states during debugging. Pygame was used for game simulation and rendering, while the time library measured computation durations to analyze performance and efficiency. These tools enabled effective development and testing of the AI algorithms.

IV. Results

The goal of our project was to build an AI solver for the 2048 game. To accomplish this, the primary objective was to design an algorithm that could consistently reach a tile with the value of 2048 in order to beat the game. With our approach, we achieved a 78% probability of beating the game. Since our algorithm hits 2048 in the majority of cases, we think we have successfully accomplished our project goal.

V. Discussion

Reflecting on the results of our project, we learned a great deal about the balance of performance and computational efficiency. By experimenting with different depth levels we saw that deeper search depths lead to more accurate and strategic results, but it hinders the computation time. This project reinforced the importance of optimizing algorithms to make smart moves without hindering speed. While other algorithms may have performed better in terms of adaptability, Expectimax proved to be a strong solution.

We also thought about how our performance would change using different AI algorithms: Minimax, Deep Q-Learning, and Supervised Learning. With each of these, we considered their strengths and limitations. Minimax is an algorithm that considers two players that would try their best to maximize their own moves while minimizing the opponents. However, this approach wouldn't work because of the single player nature of the game, where the "opponent" is a random tile that nature places. Deep Q-learning is a reinforced learning algorithm that we first considered. However, due to time constraints and our understanding of it at the time, we chose to pick Expectimax instead. In addition, deep Q-learning would require us to train a model, resulting in higher computation during the training stages, while performing better after. Supervised learning is a machine learning algorithm which finds a good algorithm by adjusting its weights using gradient descent. However, it was a bad option overall because we do not possess a large quantity of good quality data to train our model in. Overall, we chose Expectimax due to practical and performance reasons.

VI. Conclusion

2048 serves as a test case and application of computational intelligence techniques. We used Expectimax, which seemed optimal considering the randomness of tile placement and value. Through experimentation with different depths, we discovered that while deeper depths are generally more accurate it is not worth the sacrifice in computation time.

Our heuristics which focus on factors like empty tiles or value tiles helped guide our AI to make informed decisions and make the most beneficial move. This allowed our AI to achieve a 78% success rate in reaching the 2048 tile meaning we beat the game. Alternative algorithms could be implemented but Expectimax was ultimately the best choice.